Progress and Preservation

Concepts of Programming Languages

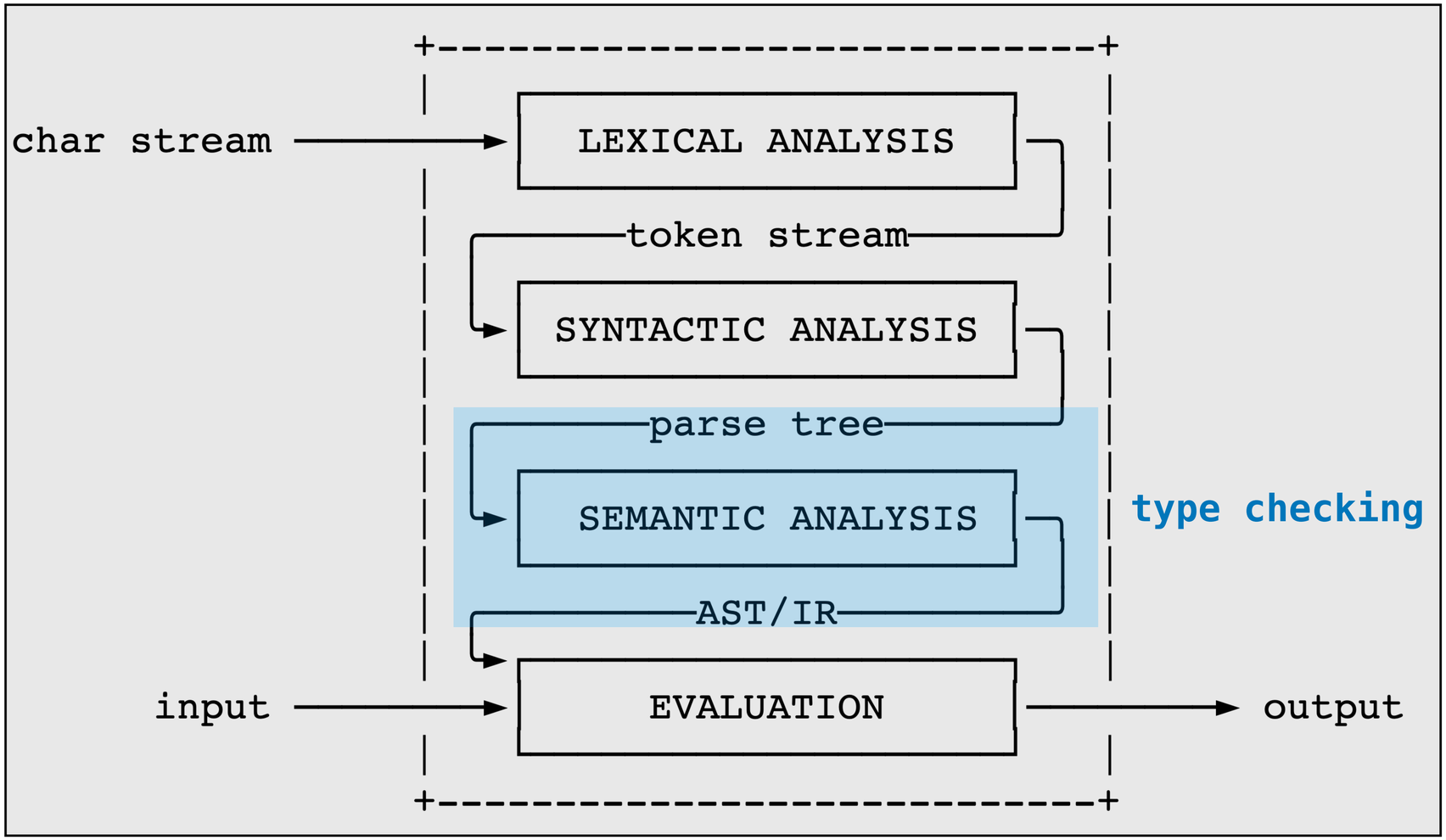
CAS CS 320

Outline

- Finish demo of STLC
- Discuss **induction** over derivations
- Show that STLC satisfies **progress** and **preservation**

Recap

Recall: The Picture



Recall: Type Checking/Inference

```
type_check : expr -> ty -> bool

type_of : expr -> ty option
```

Recall: Type Checking/Inference

```
type_check : expr -> ty -> bool

type_of : expr -> ty option
```

Type checking: determining whether an expression can be typed

Recall: Type Checking/Inference

```
type_check : expr -> ty -> bool

type_of : expr -> ty option
```

Type checking: determining whether an expression can be typed

Type inference: *synthesizing* a type for an expression

Recall: Type Checking/Inference

```
type_check : expr -> ty -> bool

type_of : expr -> ty option
```

Type checking: determining whether an expression can be typed

Type inference: *synthesizing* a type for an expression

Theoretically, these two problems can be very different. *For STLC, they are both easy*

Recall: Syntax (STLC)

$$e ::= x \mid \lambda x^{\tau}.e \mid ee$$

$$\tau ::= \bot \mid \tau \to \tau$$

Recall: Typing (STLC)

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau}\ \textbf{var} \qquad \frac{\Gamma, x : \tau \vdash e : \tau'}{\Gamma \vdash \lambda x^{\tau} . e : \tau \to \tau'}\ \textbf{fun}$$

$$\frac{\Gamma \vdash e_1 : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 e_2 : \tau'}\ \textbf{app}$$

Recall: Semantics (STLC)

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}\ \texttt{leftApp}$$

$$\frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}\ \texttt{beta}$$

Recall: Semantics (STLC)

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}\ \texttt{leftApp}$$

$$\frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}\ \texttt{beta}$$

We can use any semantics (this is small-step CBN)

Practice Problem

$$(\lambda x^{(\bot \to \bot) \to \bot} . x(\lambda z^{\bot} . x(wz)))y$$

Determine the smallest context such that the above expression is well-typed (also give its type)

Answer

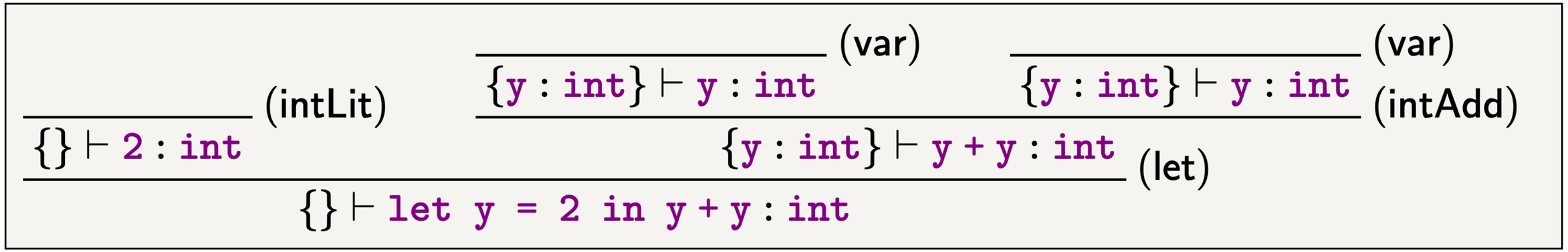
$$(\lambda x^{(\bot \to \bot) \to \bot} . x(\lambda z^{\bot} . x(wz)))y$$

demo

Induction over Derivations

The Key Idea

The Key Idea



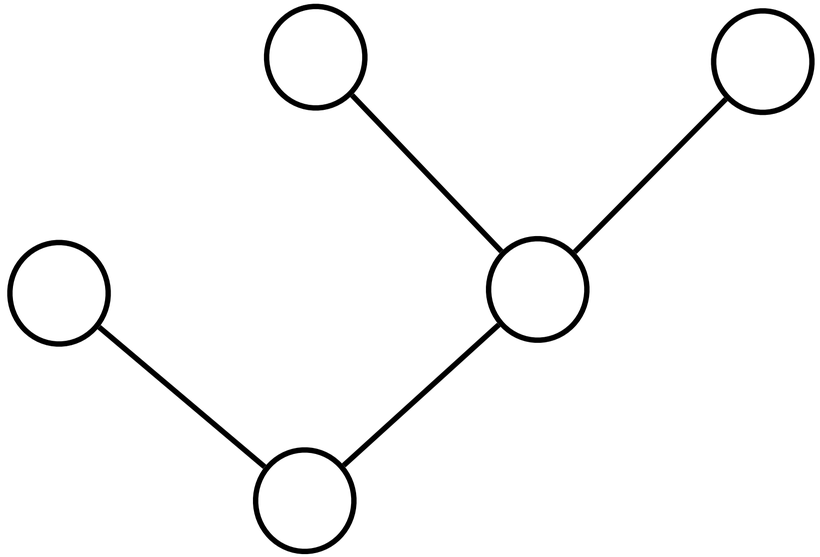
Derivations are *trees*

The Key Idea

$$\dfrac{\dfrac{}{\{\} \vdash \texttt{2} : \texttt{int}}\,(\text{intLit}) \qquad \dfrac{\dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var}) \qquad \dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var})}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} + \texttt{y} : \texttt{int}}\,(\text{intAdd})}{\{\} \vdash \texttt{let y = 2 in y + y} : \texttt{int}}\,(\text{let})$$

Derivations are *trees*

We can prove things about trees using induction



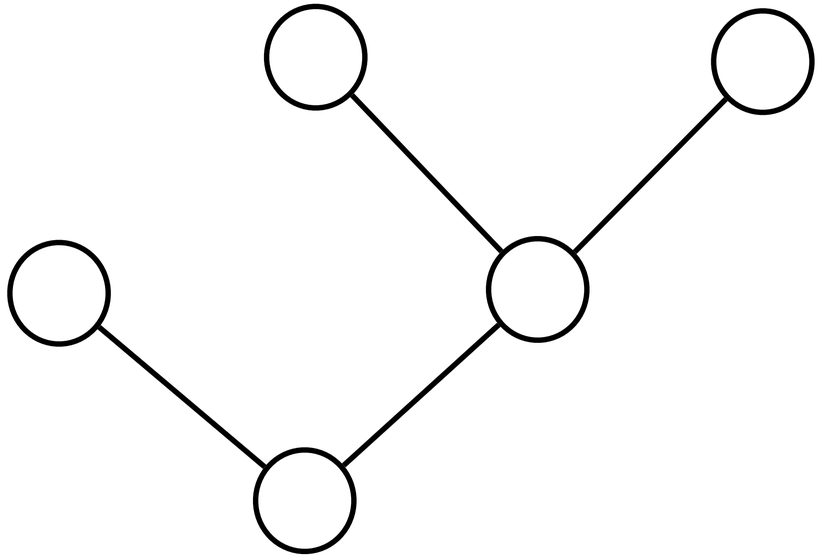
The Key Idea

$$\dfrac{\dfrac{}{\{\} \vdash \texttt{2} : \texttt{int}}\,(\text{intLit}) \qquad \dfrac{\dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var}) \qquad \dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var})}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} + \texttt{y} : \texttt{int}}\,(\text{intAdd})}{\{\} \vdash \texttt{let y = 2 in y + y} : \texttt{int}}\,(\text{let})$$

Derivations are *trees*

We can prove things about trees using induction

We can prove things about *derivable judgments* using induction



The Key Idea

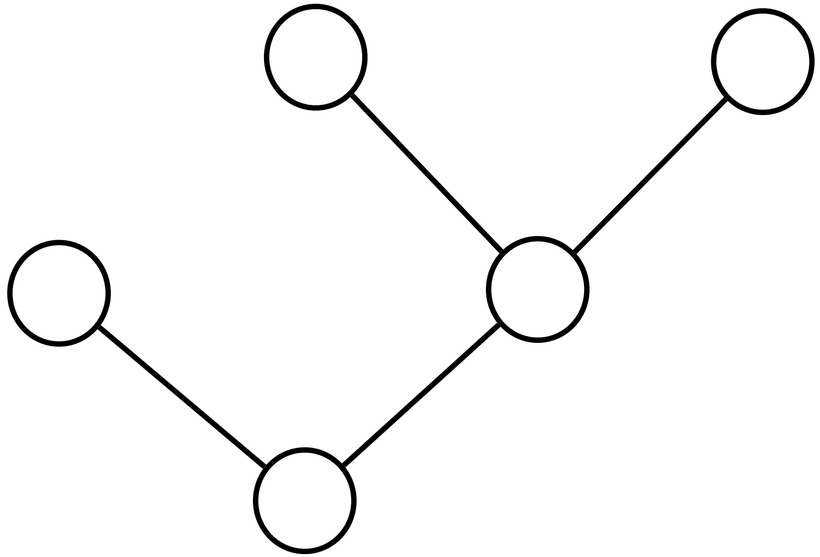
$$\dfrac{\dfrac{}{\{\} \vdash \texttt{2} : \texttt{int}}\,(\text{intLit}) \qquad \dfrac{\dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var}) \qquad \dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var})}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} + \texttt{y} : \texttt{int}}\,(\text{intAdd})}{\{\} \vdash \texttt{let y = 2 in y + y} : \texttt{int}}\,(\text{let})$$

Derivations are *trees*

We can prove things about trees using induction

We can prove things about *derivable judgments* using induction

Important: Every derivable judgment corresponds to a derivation



Induction over $\mathbb{N}$

Induction over Derivations

Warm-up: Well-Scopedness

Warm-up: Well-Scopedness

Warm-up: Well-Scopedness

An expression e is **well-scoped** with respect to a context Γ if $x \in FV(e)$ implies x appears in Γ

Warm-up: Well-Scopedness

An expression e is **well-scoped** with respect to a context Γ if $x \in FV(e)$ implies x appears in Γ

> **Theorem.** If e is well-typed in Γ, then e is well-scoped

Warm-up: Well-Scopedness

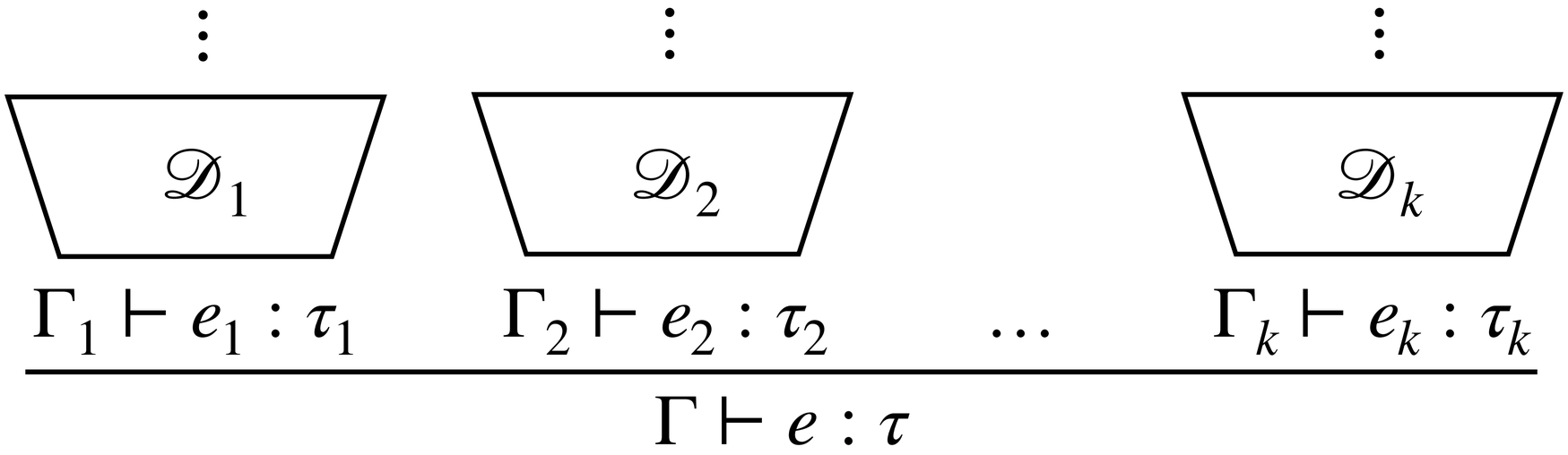
An expression e is **well-scoped** with respect to a context Γ if $x \in FV(e)$ implies x appears in Γ

> **Theorem.** If e is well-typed in Γ, then e is well-scoped

Proof. By induction on *derivations*

Base Case: Axioms

Inductive Hypothesis



$e_1, \ldots, e_k$ *are well-scoped (because they're well-typed in the each of their contexts)*

Inductive Step 1: Application

Inductive Step 2: Function

What if the last rule I applied was abstraction?

Progress and Preservation

Recall: Type Safety

Recall: Type Safety

Thm. If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash v : \tau$

Recall: Type Safety

> **Thm.** If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash v : \tau$

With small-step semantics, we can give a finer-grained analysis:

Recall: Type Safety

> **Thm.** If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash v : \tau$

With small-step semantics, we can give a finer-grained analysis:

> **Thm.**
>
> » *(progress)* If $\cdot \vdash e : \tau$ then e is a value or $e \longrightarrow e'$ for some e'
>
> » *(preservation)* If $\Gamma \vdash e : \tau$ and $e \longrightarrow e'$ then $\Gamma \vdash e' : \tau$

Recall: Type Safety

> **Thm.** If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash v : \tau$

With small-step semantics, we can give a finer-grained analysis:

> **Thm.**
>
> » *(progress)* If $\cdot \vdash e : \tau$ then e is a value or $e \longrightarrow e'$ for some e'
>
> » *(preservation)* If $\Gamma \vdash e : \tau$ and $e \longrightarrow e'$ then $\Gamma \vdash e' : \tau$

These results are *fundamental*. They tell us that our PL is well-behaved (it's a "good" PL)

Recall: Type Safety

> **Thm.** If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash v : \tau$

With small-step semantics, we can give a finer-grained analysis:

goal for today

> **Thm.**
>
> » *(progress)* If $\cdot \vdash e : \tau$ then e is a value or $e \longrightarrow e'$ for some e'
>
> » *(preservation)* If $\Gamma \vdash e : \tau$ and $e \longrightarrow e'$ then $\Gamma \vdash e' : \tau$

These results are *fundamental*. They tell us that our PL is well-behaved (it's a "good" PL)

Recall: STLC

Syntax

$$e ::= x \mid \lambda x^{\tau}.\, e \mid ee$$

$$\tau ::= \bot \mid \tau \to \tau$$

Typing

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau}\ \textbf{var} \qquad \frac{\Gamma, x : \tau \vdash e : \tau'}{\Gamma \vdash \lambda x^{\tau}.\, e : \tau \to \tau'}\ \textbf{fun}$$

$$\frac{\Gamma \vdash e_1 : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 e_2 : \tau'}\ \textbf{app}$$

Semantics

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}\ \textbf{leftApp} \qquad \frac{}{(\lambda x.\, e) e' \longrightarrow [e'/x]e}\ \textbf{beta}$$

Progress (STLC)

> **Theorem.** If e is well-typed then e is a value or there is an expression e' such that $e \longrightarrow e'$

Proof. By induction over *derivations*

Base Case: Variables

Inductive Step 1: Application

Inductive Step 2: Functions

Preservation (STLC)

Theorem. If $\Gamma \vdash e : \tau$ and $e \longrightarrow e'$ then $\Gamma \vdash e' : \tau$

Proof. By induction over *derivations*

This one is tricker...

Base Case: Vars

Inductive Step 1: Functions

Inductive Step 2: Application

Inductive Step 2.1: leftEval

Inductive Step 2.1: Beta

Substitution Lemma

Lemma. If $\Gamma \vdash e_2 : \tau_2$ and $\Gamma, x : \tau_2 \vdash e : \tau$ then

$$\Gamma \vdash [e_2/x]e : \tau$$

That is, if e is well-typed in a context with (x, τ) then we can substitute x with anything of type τ and it's still the same type

(we can prove this by, you guessed it, induction on derivations)

The Point

```
let rec eval env e =
  match e with
  | Var x -> Env.find x env
  ...
```

Progress and preservation tell us that **terms never get stuck during evaluation**

Our type system ensures we only evaluate programs that make sense.

Summary

Progress and **preservation** are fundamental features of good programming languages

We can prove things about well-typed expressions by performing **induction** over derivations